Department of Engineering Science
School of Science & Technology



CES 552 Network Architecture
Transport Layer Part II

Dr. Jingxian Wu
(707) 664-3491
Jingxian.wu@sonoma.edu

OUTLINE

- Functions & Services
- Elements of transport protocol
- **A simple transport protocol**
- Internet transport protocol: UDP
- Internet transport protocol: TCP
- Performance issues

SIMPLE PROTOCOL

- **A simple connection oriented protocol**
  - Similar to, but much simpler than TCP

- **Service primitives**
  - listen
  - connect
  - send
  - receive
  - disconnect

SIMPLE PROTOCOLS: SERVICE PRIMITIVES

- **connum = listen(local)**
  - Announces caller's willingness to accept connection
  - After listen is called, the host will be blocked until an attempt is made to connect to it.
  - local: local port number
- **connum = connect(local, remote)**
  - Tries to establish between local and remote
  - Local: local transport address
  - Remote: remote transport address
  - What if listen has not yet been executed at remote host?
    - Option 1: Connect returns an error
    - Option 2: The host calling connect is blocked until listen is executed
    - Option 3: The host calling connect wait for some time, then returns an error message.

SIMPLE PROTOCOLS: SERVICE PRIMITIVES

- **Status = send(connum, buffer, bytes)**
  - Send data to remote host through the connection specified by connum.
  - Buffer: storing the data to be sent
  - Bytes: how many bytes will be sent.
- **Status = receive(connum, buffer, bytes)**
  - Indicates the caller's desire to accept data through the connection
  - Buffer: the place to store data
  - Bytes: how many bytes will be retrieved.
- **status = disconnect(connum)**
  - Disconnect the connection specified by connum.
  - Symmetric disconnection

SIMPLE PROTOCOL

- **Assumptions:**
  - To simplify the code for transport entities, use connection-oriented reliable services from network layer
  - Service primitives from network layer
    - To_net: send packet to network layer
    - From_net: retrieve packet from network layer
    - Sleep: After sleep has been called, the transport entity will stop until a new packet arrives at network layer or time out
- **Elements of the transport layer**
  - Addressing
  - Connection setup (two way hand-shaking)
  - Connection release (two way hand-shaking)
  - Flow control
    - Tx CANNOT send until get permission (credit) from Rx
    - 1 packet per credit.

SIMPLE PROTOCOLS: TRANSPORT ENTITIES

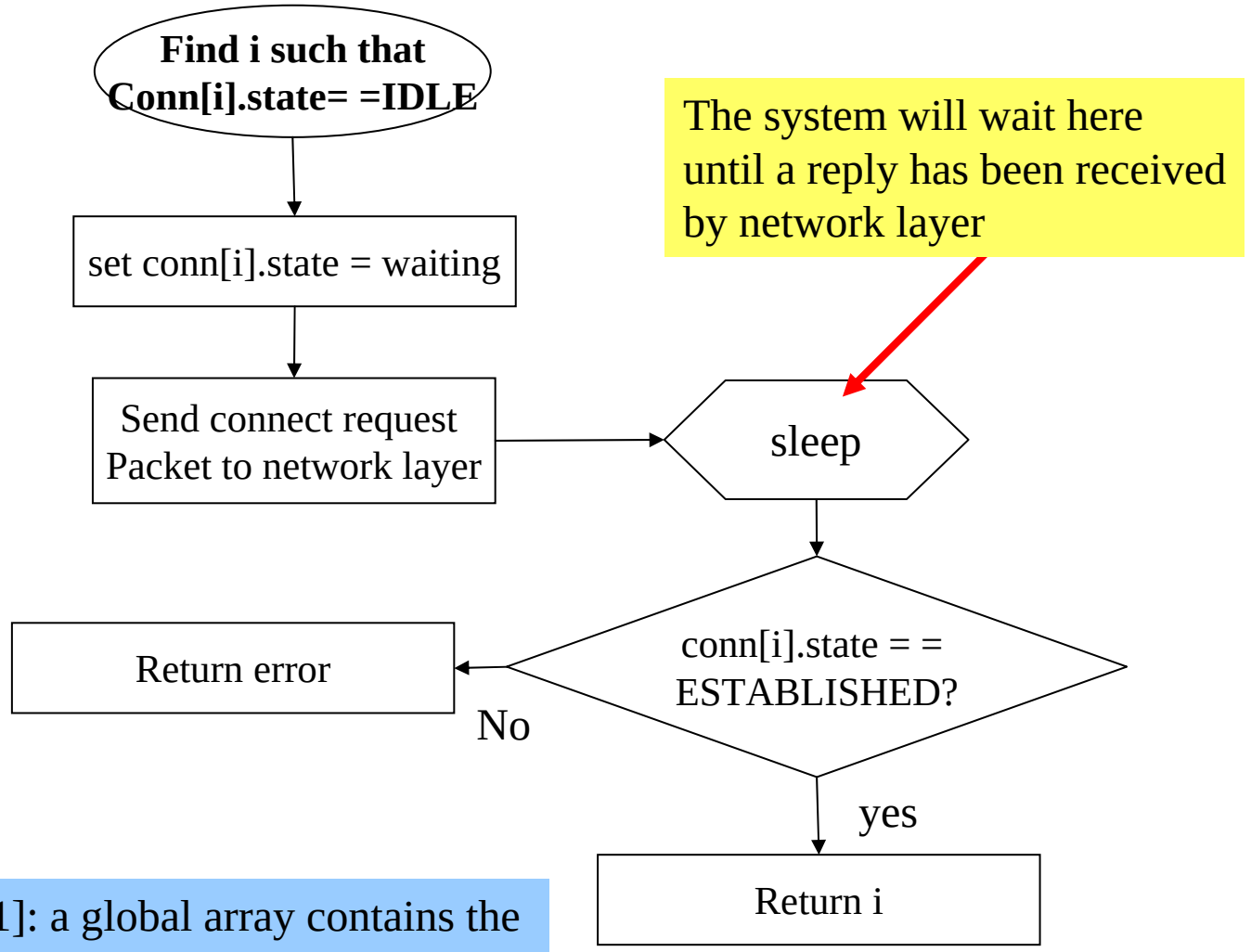
- **Connection status:**
  - IDLE: connection not established yet
  - WAITING: connect has been executed, and connection request packet has been sent to remote host
  - QUEUED: a connection request has been received, no listen yet.
  - ESTABLISHED: the connection has been established
  - SENDING: the user is waiting for permission to send a packet
  - RECEIVING: a receive has been executed
  - DISCONNECTING: a disconnect has been executed

SIMPLE PROTOCOLS: TRANSPORT ENTITIES

- **Data structure: struct conn**
  - transport_address local_address, remote_address
  - cstate state; /* one of the seven states */
  - unsigned char * user_buf_addr; /* pointer to receive buffer */
  - int byte_count; /* how many bytes have been Tx/Rx so far */
  - int clr_req_received; /* set when CLEAR_REQ packet received*/
  - int timer; /* time out request packet */
  - int credits; /* number of packets might be sent: flow control */
- **Global variable**
  - Struct conn conn[MAX_CONN+1];
    - The array contains the information of all connections.
    - The contents can be set/viewed by both network layers, transport layers on server and client
    - Serves as a service interface from network layer
- **Tanenbaum's book, p. 518 – 521**

SIMPLE PROTOCOLS: TRANSPORT ENTITIES

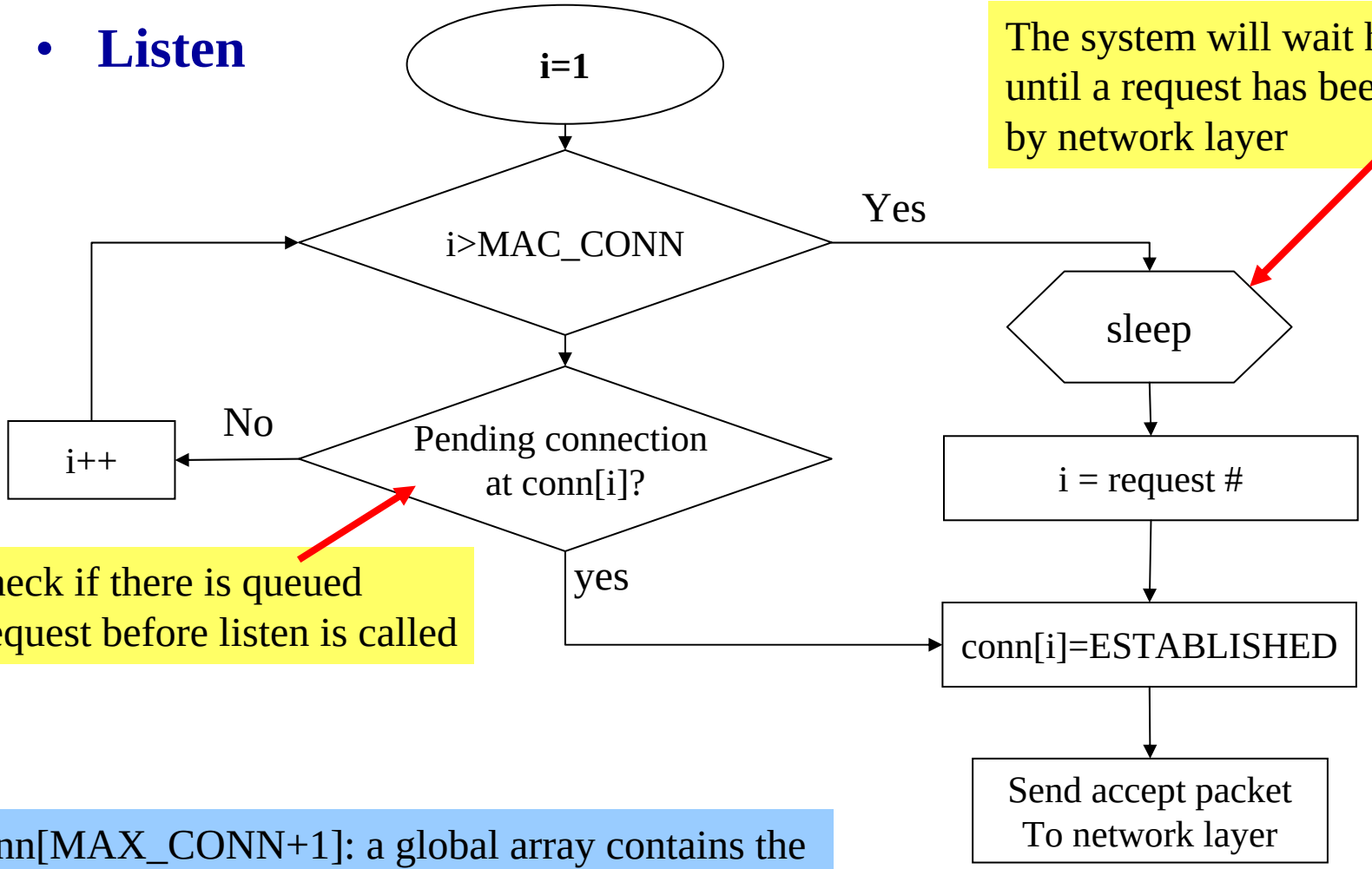
- **connect**



conn[MAX_CONN+1]: a global array contains the information of all potential connections

SIMPLE PROTOCOLS: TRANSPORT ENTITIES

- **Listen**



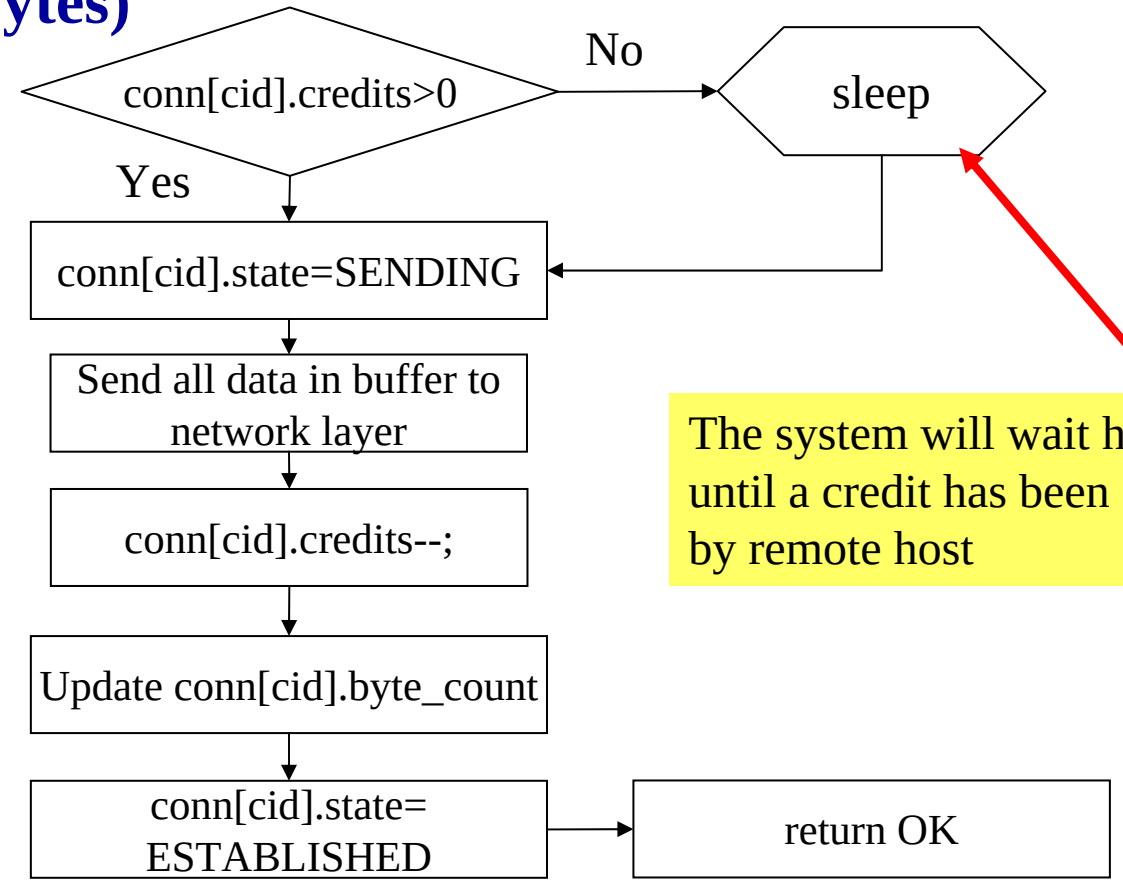
The system will wait here until a request has been received by network layer

Check if there is queued Request before listen is called

conn[MAX_CONN+1]: a global array contains the information of all potential connections

SIMPLE PROTOCOLS: TRANSPORT ENTITIES

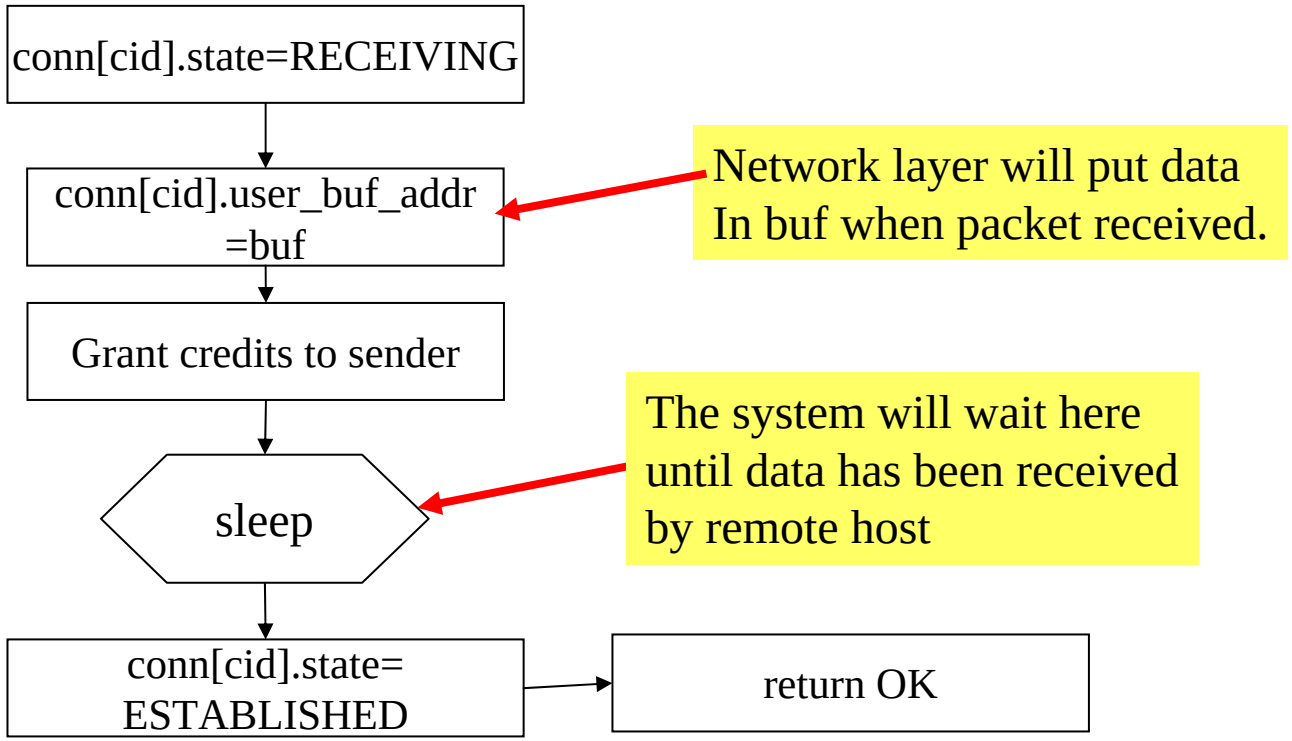
- **send(cid, buffer, bytes)**



Credit is for flow control purpose

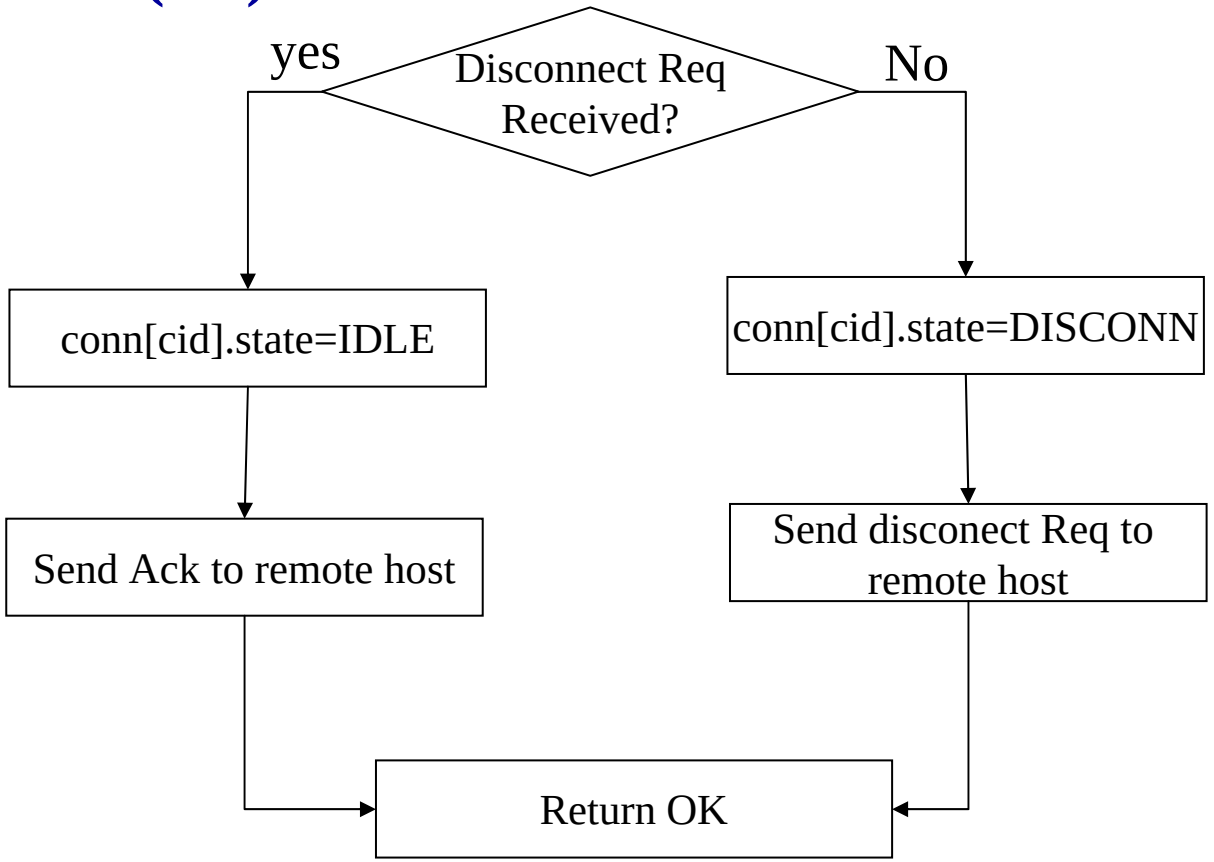
SIMPLE PROTOCOLS: TRANSPORT ENTITIES

- **receive(cid, buf, bytes)**



SIMPLE PROTOCOLS: TRANSPORT ENTITIES

- **disconnect(cid)**



OUTLINE

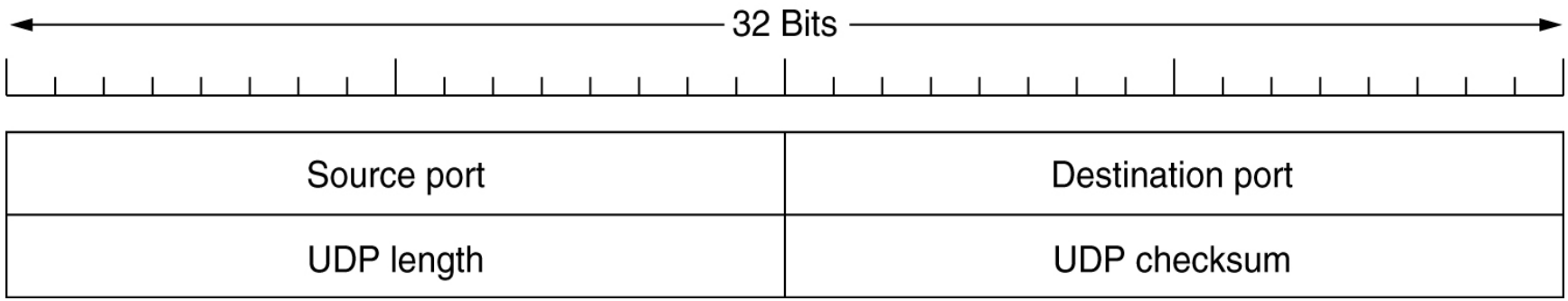
- Functions & Services
- Elements of transport protocol
- A simple transport protocol
- **Internet transport protocol: UDP**
- Internet transport protocol: TCP
- Performance issues

UDP

- **User datagram protocol (UDP) – RFC 768**
  - Connectionless protocol
  - Simple: IP with a short header
  - Fewer message required (no connection setup, acknowledgement).
  - Example: DNS (Domain Name System)
    - Client send a request contains a domain name
    - DNS server replies the IP of the domain name
    - Only two messages.
  - Good for short transaction, or delay sensitive applications.
- **Why UDP? Why not use IP directly?**
  - Each host has only one IP address
  - Multiple network applications are running simultaneously.
  - One UDP entity can provide services to multiple users.
    - Each user has it's own transport port #.
  - We can easily identify the destination application by using
    - (IP, UDP port #)

We want to use the addressing element of UDP.

UDP

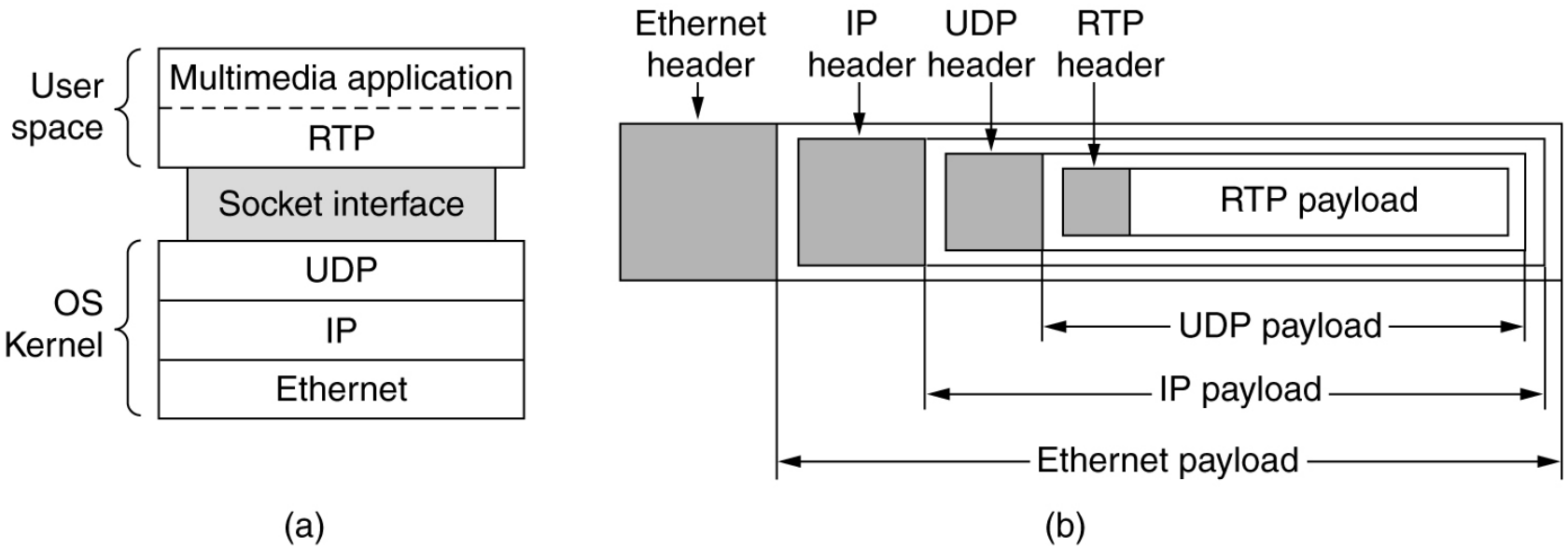


- **Header**
  - Source port: port # of source
  - Destination port: port # of destination
  - Length: the length of the entire UDP packet (head + payload).
  - Checksum: optional. All 0s if not computed.
- **What the UDP does not do**
  - Flow control
  - Acknowledgement
  - Retransmission (retransmission is meaningless for real-time application)

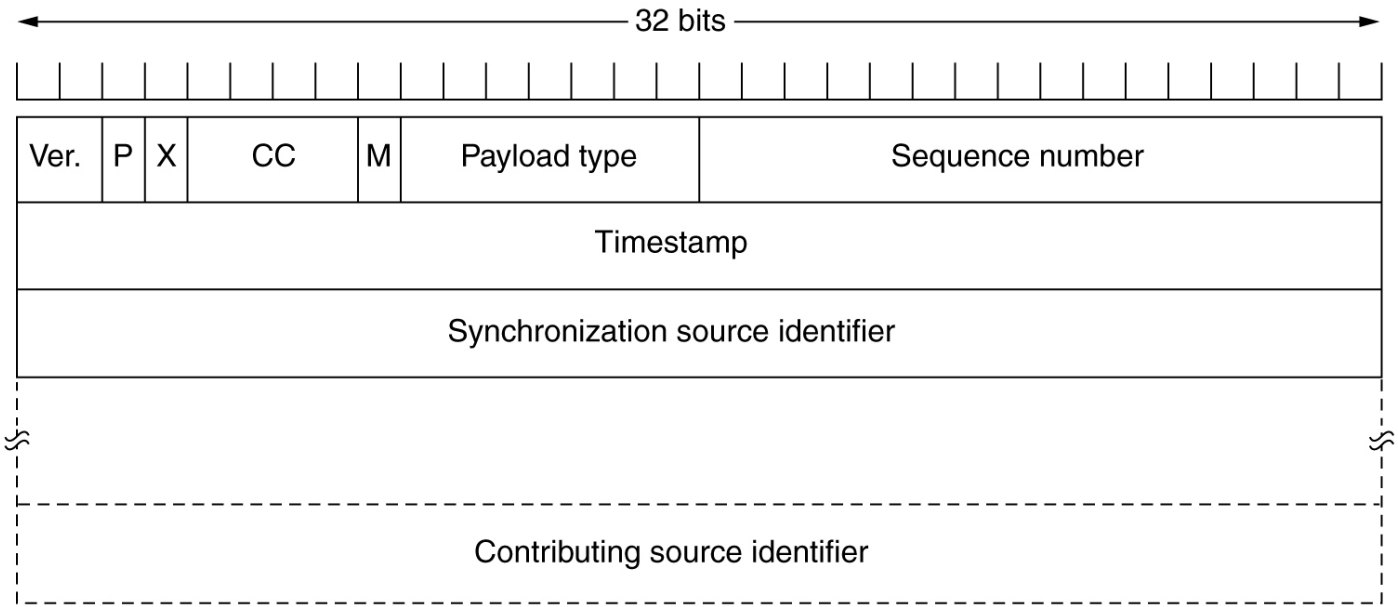
These operations are left to the applications!

UDP: REAL-TIME TRANSPORT PROTOCOL

- **Real-time transport protocol (RTP)**
  - The engine for real-time applications
    - Internet telephony, videoconferencing, video-on-demand, etc.

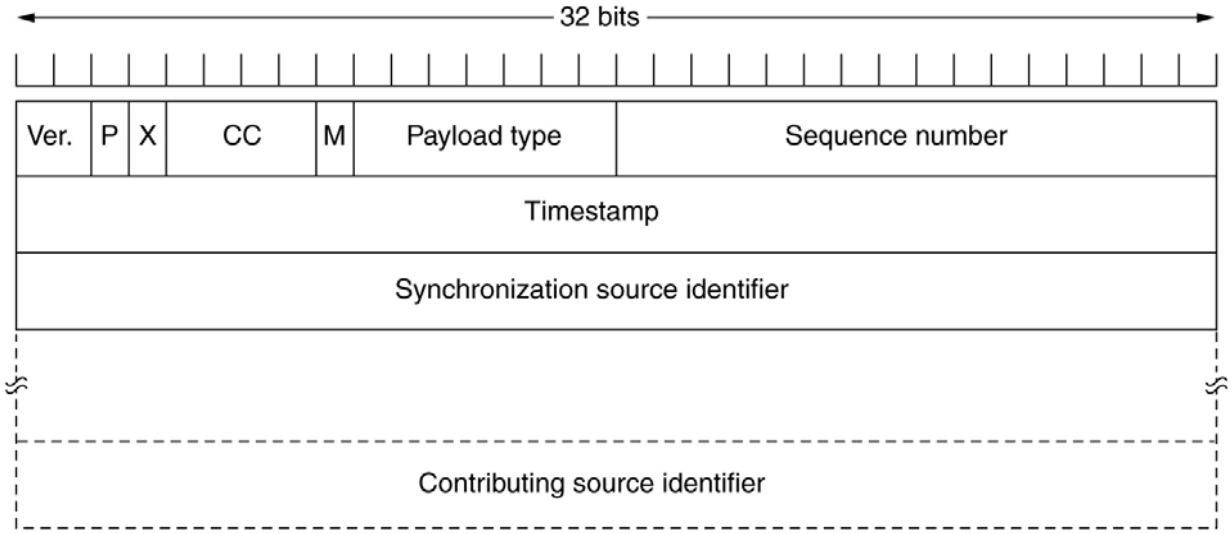


UDP: REAL-TIME TRANSPORT PROTOCOL



- Ver: version. Current: 2
- P: indicates that the packet has been padded to a multiple of 32-bit
- X: indicates the existence of extension header (for future purpose)
- CC: how many streams are multiplexed for one application.
  - e.g. movie = video + soundtrack 1 (English) + soundtrack 2 (Spanish)
- M: application marker. Interpreted by applications!
  - E.g. start of a video frame, or start of a word in audio

UDP: REAL-TIME TRANSPORT PROTOCOL



- Payload type: what kind of encoding is in the payload field?
  - E.g. PCM, MP3, MPEG2 (encode/decode are performed by application)
- Sequence number: incremented on each RTP packet
  - Only used to detect lost packet. No flow control. No Ack. No ReTx.
- Timestamp: relative to the start of the stream.
  - Allow the rx to playback the frame at the right time ➔ Reduce jitter!
- Synchronization source ID: if multiple applications are multiplexed into one UDP stream (upward multiplexing), this tells which application this packet belongs to.
- Contributing source ID: the ID of the stream for an application (e.g. soundtrack 1 of a movie)

UDP: REAL-TIME TRANSPORT PROTOCOL

- **Properties of RTP**
  - Provide basic functions required by most real-time applications.
    - E.g. timestamp, payload type
  - Multiplex several real-time data streams onto a single stream of UDP packets.
  - No flow control, No acknowledgement, No retransmission.
    - Lost packet will be taken care of by applications
      - E.g. guess the contents of lost packet by looking at its previous packet and following packet.
  - Doesn't provide any mechanism for QoS.

UDP: REAL-TIME TRANSPORT PROTOCOL

- **Real-time transport control protocol (RTCP)**
  - Transmit control information for RTP.
  - E.g. Rx feedbacks delay, jitter, bandwidth, congestion and other properties to the source
    - The source application will adjust the coding rate based on information.
      - Higher coding rate ➔ less compression ➔ better display quality
      - Higher coding rate ➔ less compression ➔ larger bandwidth
  - Synchronization between different streams (e.g. video and audio)
  - Provides text description of the contents.
    - E.g. title of the movie, singer, lyrics

OUTLINE

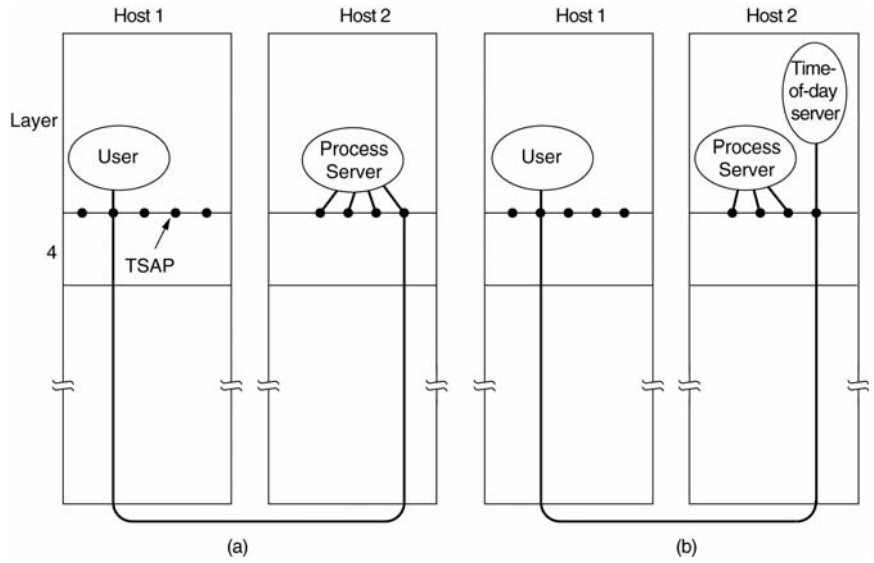
- Functions & Services
- Elements of transport protocol
- A simple transport protocol
- Internet transport protocol: UDP
- **Internet transport protocol: TCP**
- Performance issues

TCP: SERVICES

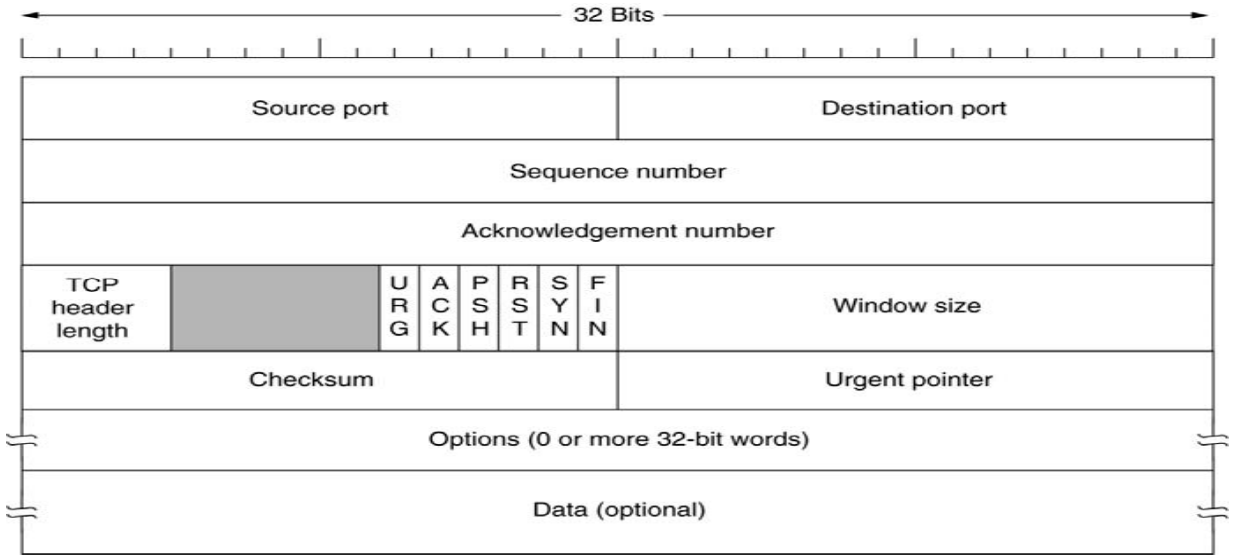
- **Transmission control protocol – RFC 793, 1122, 1323**
  - Reliable end-to-end byte stream over an unreliable internetwork
    - IP gives no guarantee of the delivery of datagram
    - It's up to TCP to guarantee the in-order and reliable delivery of the data to application layer.
- **TCP service model**
  - Application access the services provided by TCP by creating sockets at both communication parties.
    - Each socket is associated with a unique port number.
  - Connections are established between sockets.
    - One socket might be used for multiple connections.
      - E.g. several users can connect to the same FTP server simultaneously.
  - Byte stream: all bytes are treated equally.
  - Full duplex: data transmissions can occur at both directions simultaneously.

TCP: SERVICES

- **Daemon**
  - Server programs runs at background
    - FTP daemon is associated with port 21
  - Internet daemon (inetd)
    - Attached to multiple ports and waiting for connection requests.
    - When a connection request for a particular application comes in, inetd will fork off a new process and wake up the corresponding service daemon.

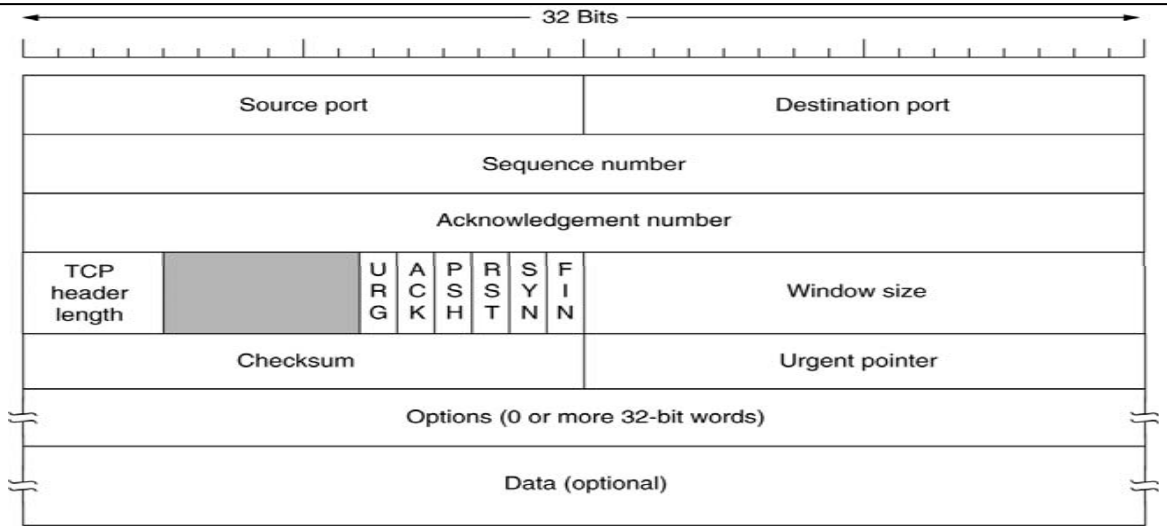


TCP: PROTOCOL



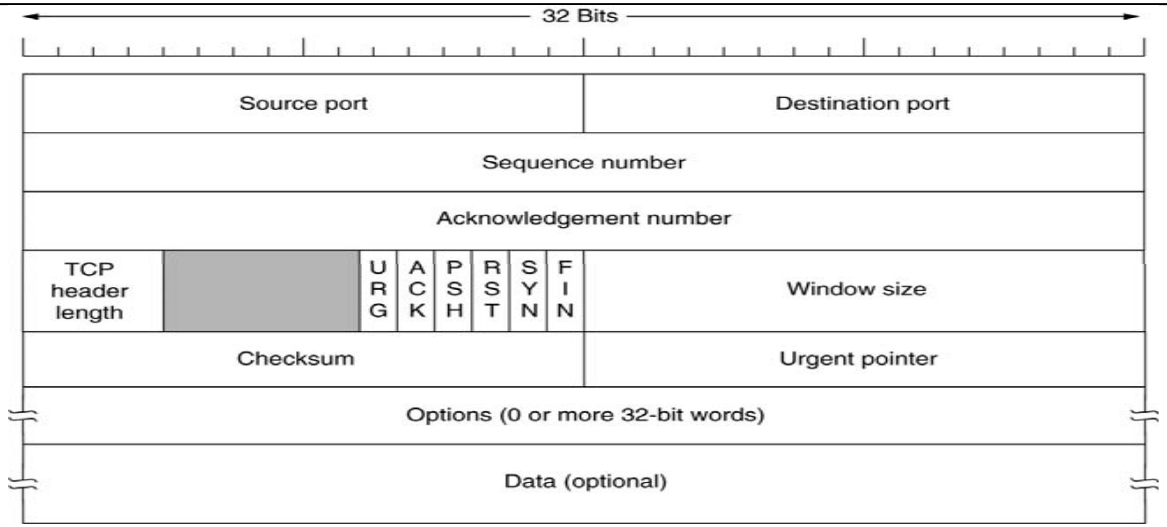
- Source port, destination port
  - 16-bit: $2^{16} = 65{,}536$
  - Specify the sockets on source and destination.
- Sequence number: $2^{32} \approx 4$ GB
  - For a particular connection, each byte has its own sequence number
  - The sequence number of the first byte in the payload
- Acknowledge number: (Used in combination with the ACK flag)
  - All bytes (0, Ack# - 1) have been successfully received
  - The next expected byte is Ack#.

TCP: PROTOCOL



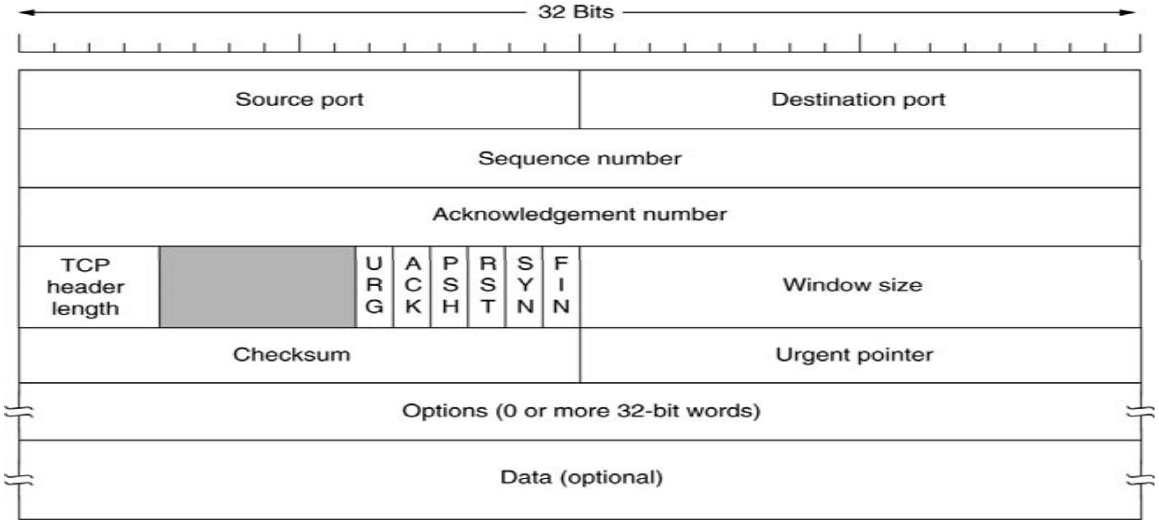
- TCP header length (in the unit of 32-bit word)
  - How many 32-bit words are in the TCP header.
  - Fixed header: 20 bytes. Optional header: variable length.
- URG: urgent flag, used in combination with urgent pointer
  - If set to 1, indicating the urgent pointer is in use
- Urgent pointer: (relative offset from the seq#)
  - pointing to a byte in the payload starting from which the data is urgent
  - TCP will be notified by application which data is urgent
    - E.g. telnet, ctrl+C to terminate an application.
  - If the data is urgent, TCP cannot hold it in buffer. Must deliver it immediately even if the sender window is 0.

TCP: PROTOCOL



- Ack: when set to 1, the Ack# is valid. Otherwise Ack# will be ignored.
- PSH: PUSH flag
  - When set to 1, notify TCP entity to send out data immediately.
  - E.g. in remote login, hit enter.
- RST:
  - when set to 1, reset a connection (due to crash or other reasons).
  - Also used to reject invalid TCP packets (e.g. delayed duplicate).
- SYN: used for connection establishment
  - Connection request: SYN = 1, ACK = 0.
  - Connection accept: SYN = 1, ACK = 1.

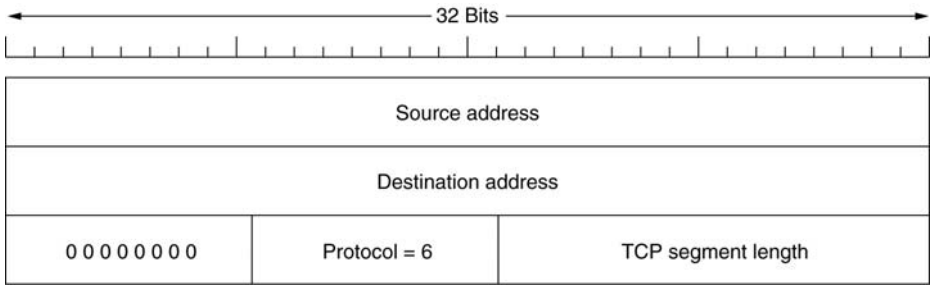
TCP: PROTOCOL



- FIN: set to 1 used for connection release.
  - After sending out a packet with FIN = 1, the host will no longer transmit more data. (but will continue receive data)
- Window size: credits for more transmission (flow control: variable size sliding window).
  - Host on the other side are allowed to transmit bytes with seq# in the range (Ack#, Ack# + window size-1).
  - Maximum credits per Ack: $2^{16} = 65{,}536 = 64KB$
  - For high speed line, this size is too small ➔ not efficient.

TCP: PROTOCOL

– Checksum
  - checks TCP header, data, and part of the IP header (source IP, dest. IP, protocol (6), TCP packet length) ➔ violate the layered structure
  - Other than TCP header, the following fields are checked.



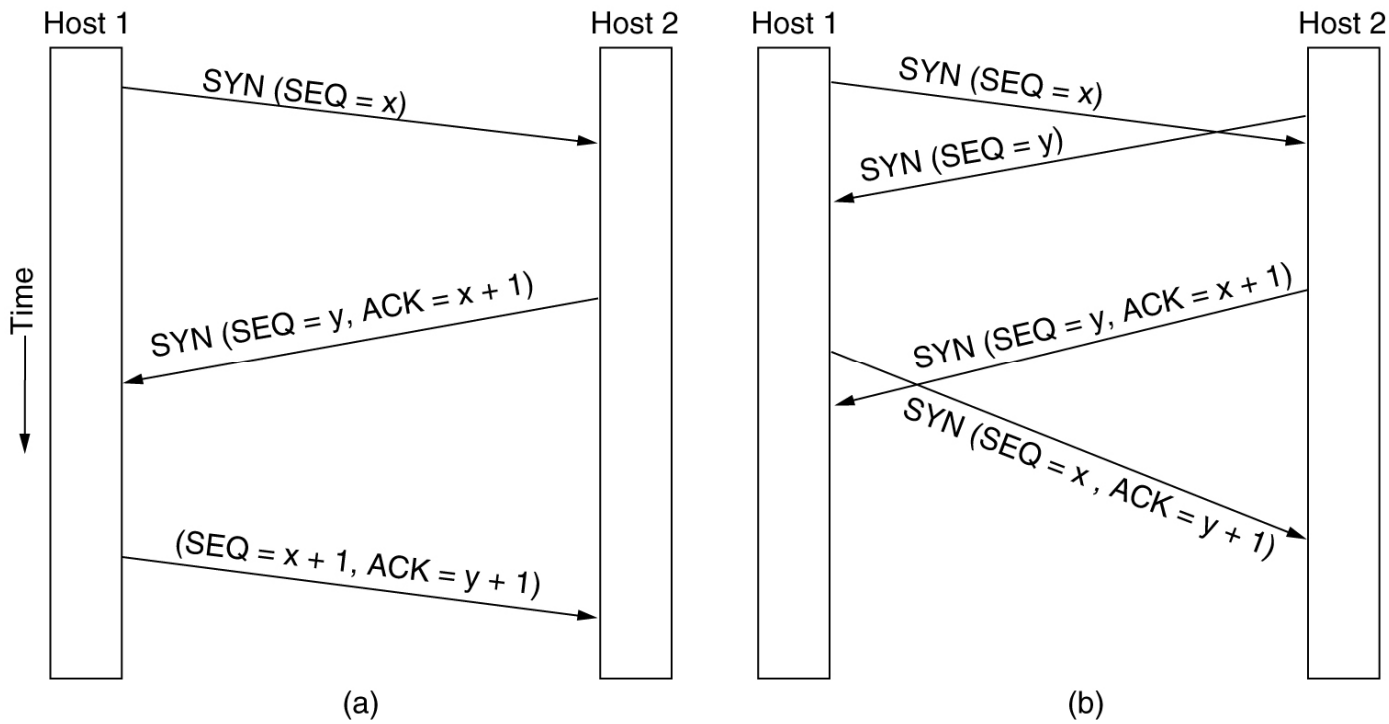
TCP: PROTOCOL

- **TCP header Options:**
  - During connection set-up, specify the maximum TCP payload it's willing to accept. (if not specified, default is 536 bytes)
    - Maximum IP payload: 65,515 bytes
    - MTU: maximum transfer unit for a certain network
      - E.g. Ethernet 1500 bytes
  - Window scale option, multiply the 64KB maximum window size with a factor
    - Large window, more efficient for high speed lines
    - Allow maximum 1GB of credits/Ack
  - Selective repeat or Go back N
    - Go back N: if one frame is in error, all frames after that frame will be reTx (default for TCP)
    - Selective repeat: only reTx the one frame that is in error

TCP: CONNECTION ESTABLISHMENT

- **Service primitives**
  - Server: LISTEN, ACCEPT
  - Client: CONNECT
- **Three-way handshake**
  - After CONNECT is called, a connection request packet will be generated.
    - Connection request packet (SYN = 1, ACK =0, Seq# = x).
    - When the connection request packet arrives at the receiver, the receiver checks if LISTEN and ACCEPT have been executed for the port.
    - If server accepts request, send back Ack (SYN = 1, ACK=1, Ack# = x+1, Seq# = y)
      - Client sends back an Ack (SYN=0, ACK=1, Ack#=y+1) to finish the three way handshake.
    - If server rejects request, send back Rej (RST = 1, ACK = 1, Ack#=x+1)
    - The initial sequence# (x, y) are assigned based on the clock at the hosts.

TCP: CONNECTION ESTABLISHMENT



If two hosts attempt to establish
A connection simultaneously,
Only one connection will be established.

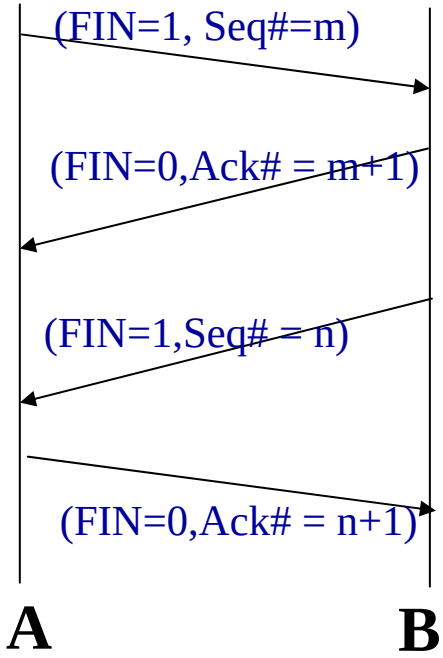
TCP: CONNECTION RELEASE

- **Full duplex → two simplex.**
- **Each of the two communication directions are released individually**

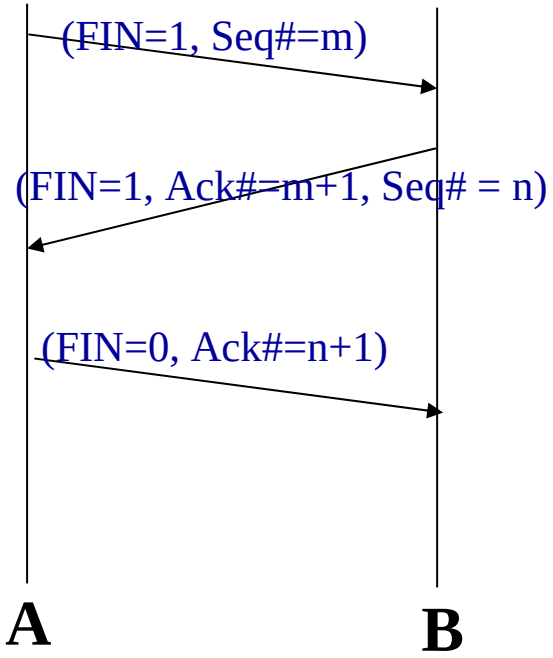
A ←→ B

  - A calls CLOSE → A will no longer send data. But A will continue to receive data.
    - A send out connection release packet (FIN = 1, Seq# = m)
    - B receives the release request. If B has more data to transmit, B will simply Ack the release request (FIN = 0, ACK=1, Ack# = m+1).
    - After A recevies B's Ack, the direction A → B is released.
      - However, A ← B is still active
  - When B wants to close the direction A ← B, B will send out release request (FIN = 1, Seq# = n).
    - A replies with (FIN = 0, ACK=1, ACK# = n+1)
    - After B receives A's Ack, the direction B → A is released.

TCP: CONNECTION RELEASE

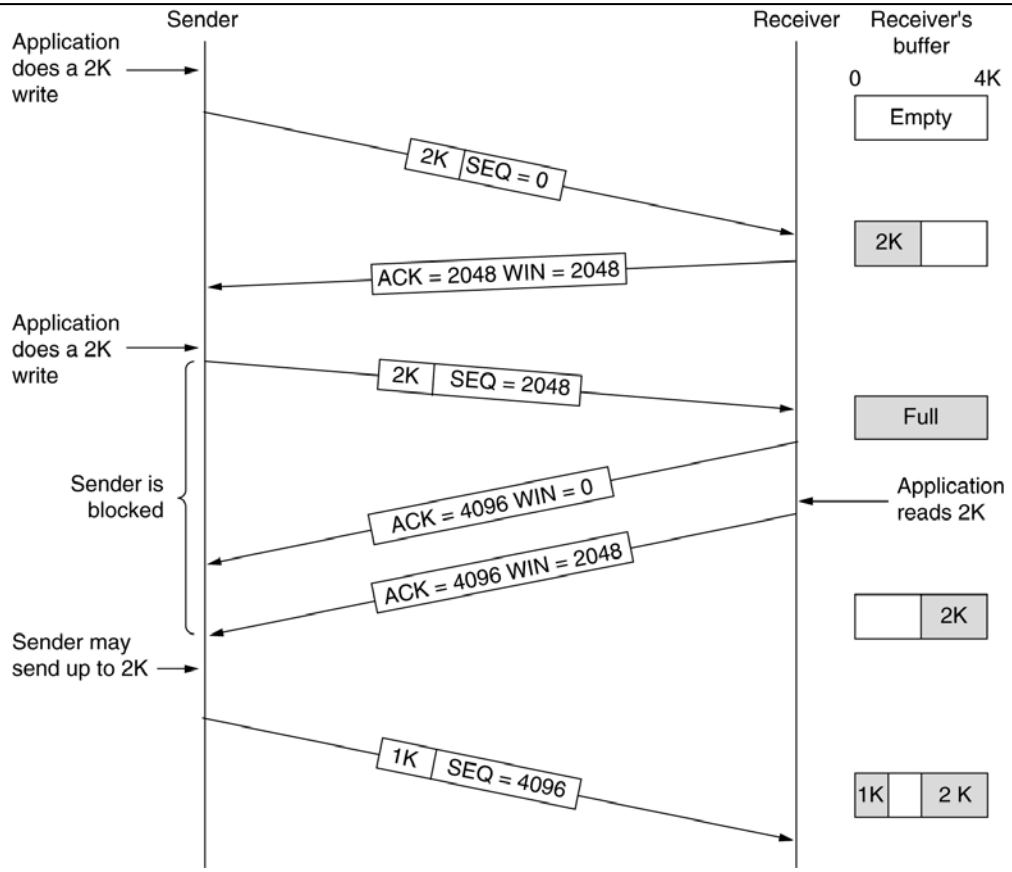


Four packets used for connection release



Three packets used for connection release

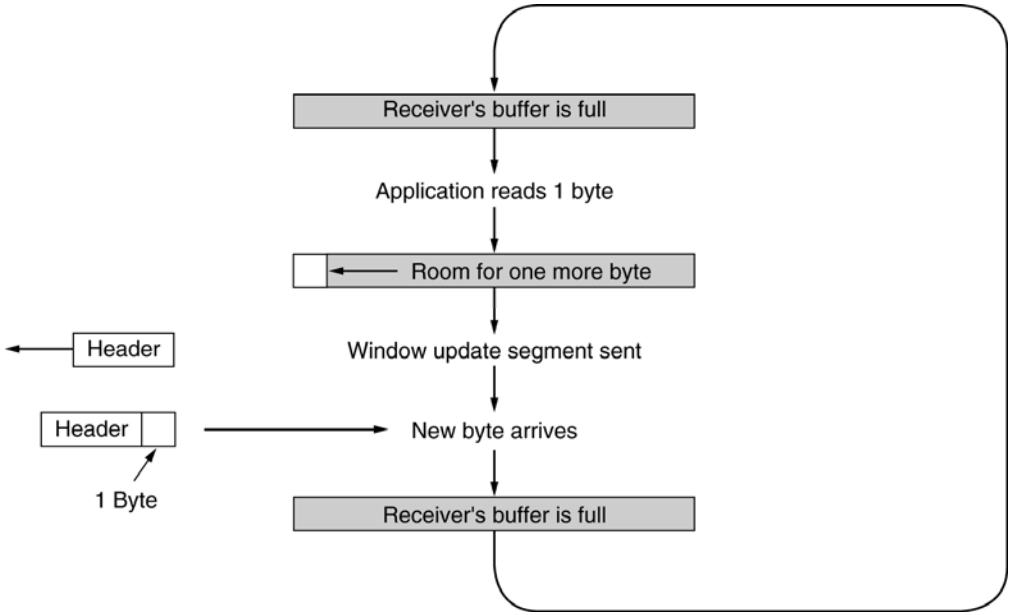
TCP: FLOW CONTROL



- **When available credit is 0, the sender can still send two kinds of data:**
  - Urgent data
  - A 1-byte packet asking for more credits.
    - To prevent deadlock.

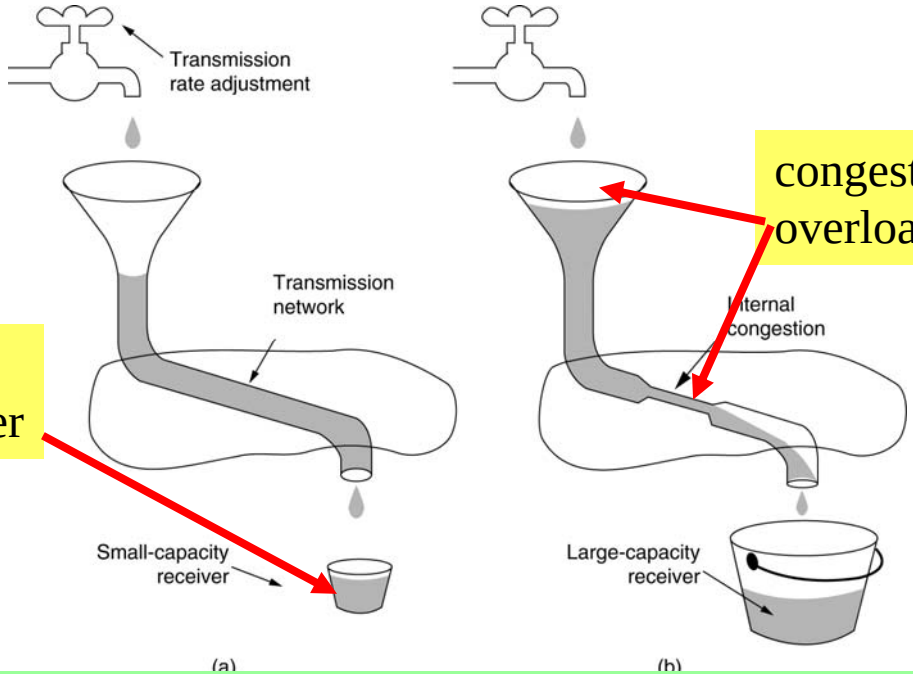
TCP: FLOW CONTROL

- **Silly window syndrome**
  - Rx grants new credits to Tx whenever 1 byte free space is available in buffer ➔ highly inefficient.
  - Solution:
    - Receiver: receiver only grants new credits to sender when it has a decent amount of space (>> 1 byte) in buffer.
    - Sender: avoid sending packets with very small payload.



TCP: CONGESTION CONTROL

- **Congestion:**
  - Occurs when the load offered in the network is more than it can handle.
    - Congestion occurs in the network.
  - Flow control: avoid buffer overflow at receiver (occurs in the host!)
- **Congestion control: avoid overloading the network**



congestion control: avoid overloading network

flow control: avoid overloading receiver

We need both flow control and congestion control! ➔ slow down the sender!

TCP: CONGESTION CONTROL

- **Congestion control and flow control are treated separately.**
  - Flow control: receiver define transmission window to make sure data size from tx < available buffer size at receiver. (window_1)
  - Congestion control: sender maintains another window by itself to avoid overload the network. (window_2)
  - Actual transmission size < min(window_1, window_2) for both flow control and congestion control
- **E.g.**
  - Receiver grants transmitter credits for 100 KB ➔ window_1 = 100 KB
  - Transmitter knows that burst over 50 KB will overload network ➔ window_2 = 50 KB
  - Transmitter will only send out 50 KB, even though the receiver allows it to transmit 100 KB.

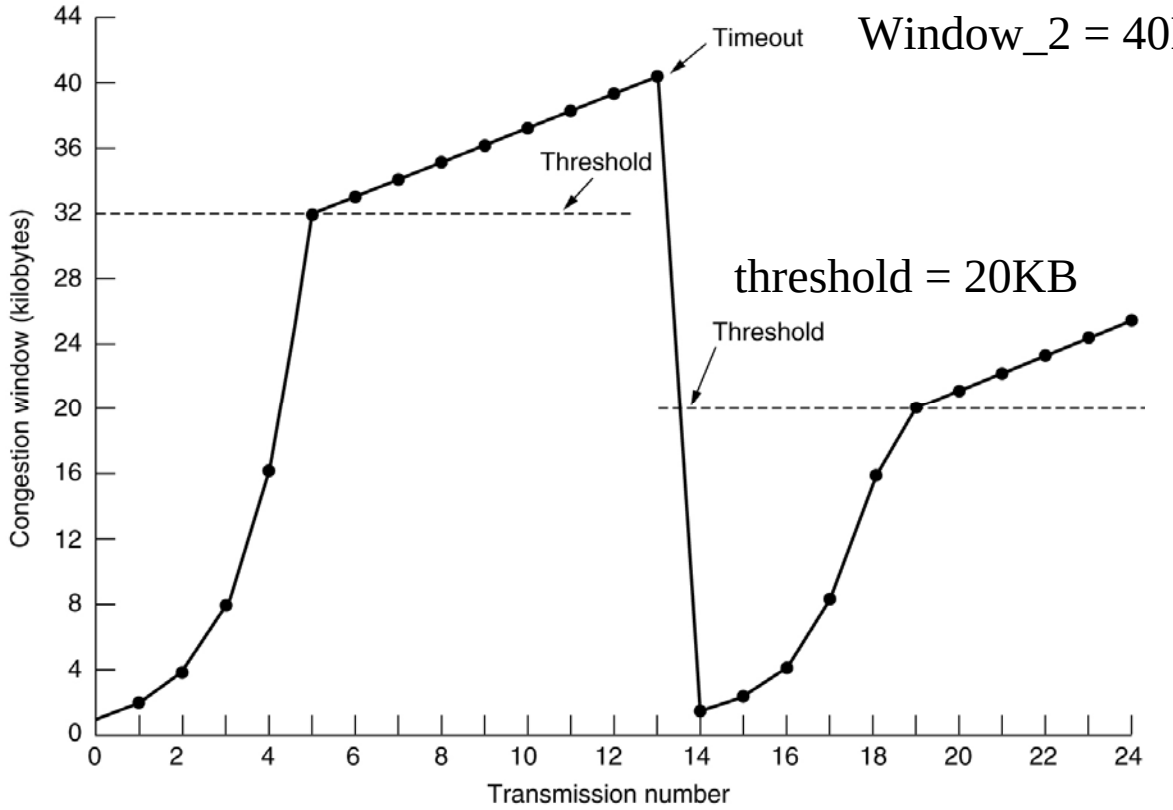
TCP: CONGESTION CONTROL

- **How could transmitter know the size of window_2?**
  - Initially, Window_2 = N. (N: a number, e.g. 1200 bytes)
  - If the first N bytes are Acked before time out, window_2 = N x 2
  - If all the N x 2 bytes are Acked before time out, window_2 = N x 2 x2.
  - If all previous bytes are Acked before time out, window_2 = window_2 x 2.
  - Size of window_2 increases exponentially if everything is fine.

  - If timer expires before Ack
    - Threshold = window_2/2;
    - window_2 = N.
  - Successful transmission
    - If window_2 < threshold, then window_2 = window_2 x 2
    - If window_2 > threshold, then window_2 = window_2 + N.
      - Linear increase after threshold

Assumption: time out is caused by congestion.

TCP: CONGESTION CONTROL



***N* = 1KB**

TCP: TIMER MANAGEMENT

- **Retransmission timer**
  - If reTx timer expires before Ack ➔ reTx
- **How to set the value of the timer?**
  - Transmission delay is highly unpredictable due to network congestion.
  - If timer is too short ➔ many timeouts ➔ unnecessary reTx.
  - If timer is too long ➔ longer transmission delay whenever a packet is lost.

  - Solution: use a highly dynamic timer value based on continuous measurement of the network performance.

TCP: TIMER MANAGEMENT

- **Dynamic timer value calculation**

$$RTT = \alpha \times RTT + (1-\alpha)M$$

Round trip delay estimate

Constant, typically 7/8

Actual round trip delay of the last packet

$$D = \alpha \times D + (1-\alpha)\,|\,RTT - M\,|$$

measures how accurate the RTT is.

$$Timeout = RTT + 4 \times D$$

TCP: TIMER MANAGEMENT

- **Other timers**
  - Persistence timer
    - Receiver grants credits to sender. However the packet get lost.
    - Sender doesn't transmit because no credit is available.
    - Receiver waiting for sender to Tx. ➔ deadlock
    - Sender has a persistence timer.
      - If this timer expires, sender will send a probe packet to Rx asking for credits.
  - Keepalive timer
    - If the connection is idle for a long time, the keepaliver timer will expire.
    - One side sends a packet to see if the other side is still there.
      - If no response, connection will be released.

OUTLINE

- Functions & Services
- Elements of transport protocol
- A simple transport protocol
- Internet transport protocol: UDP
- Internet transport protocol: TCP
- **Performance issues**

PERFORMANCE

- **Network performance metrics**
  - Throughput (bits/sec)
    - How many bits can be transmitted in a unit time.
  - Delay (sec)
    - How long will it take to deliver a packet from source to destination.
  - Jitter (sec)
    - The variation in delay (e.g. standard deviation of delay)
  - Packet loss rate
    - How many packets are lost (or in error) during transmission.

PERFORMANCE

- **Bandwidth (Raw data rate)**
  - Maximum data transmission rate at hardware level
  - Determined by signaling rate and network interface card (physical layer metric)
  - Measures how fast data can be put in the channel
  - Measured on physical layer.
- **Throughput**
  - Maximum transmission rate in the software level. – (# of bits/time)
  - Time: software processing time + actual transmission time.
  - # of bits: Overhead from protocols + data
  - Measured on the transport layer
- **Goodput**
  - Maximum reliable data transmission rate. – (# of bits/time)
  - Time: software processing time + actual transmission time + retransmission time
    - Effects of recovery from error or lost packet is accounted (e.g. retransmission)
  - # of bits: only pure data from application
    - Overhead from protocols is not counted during the calculatio.
  - "reliable throughput"
  - Measured on the application layer.

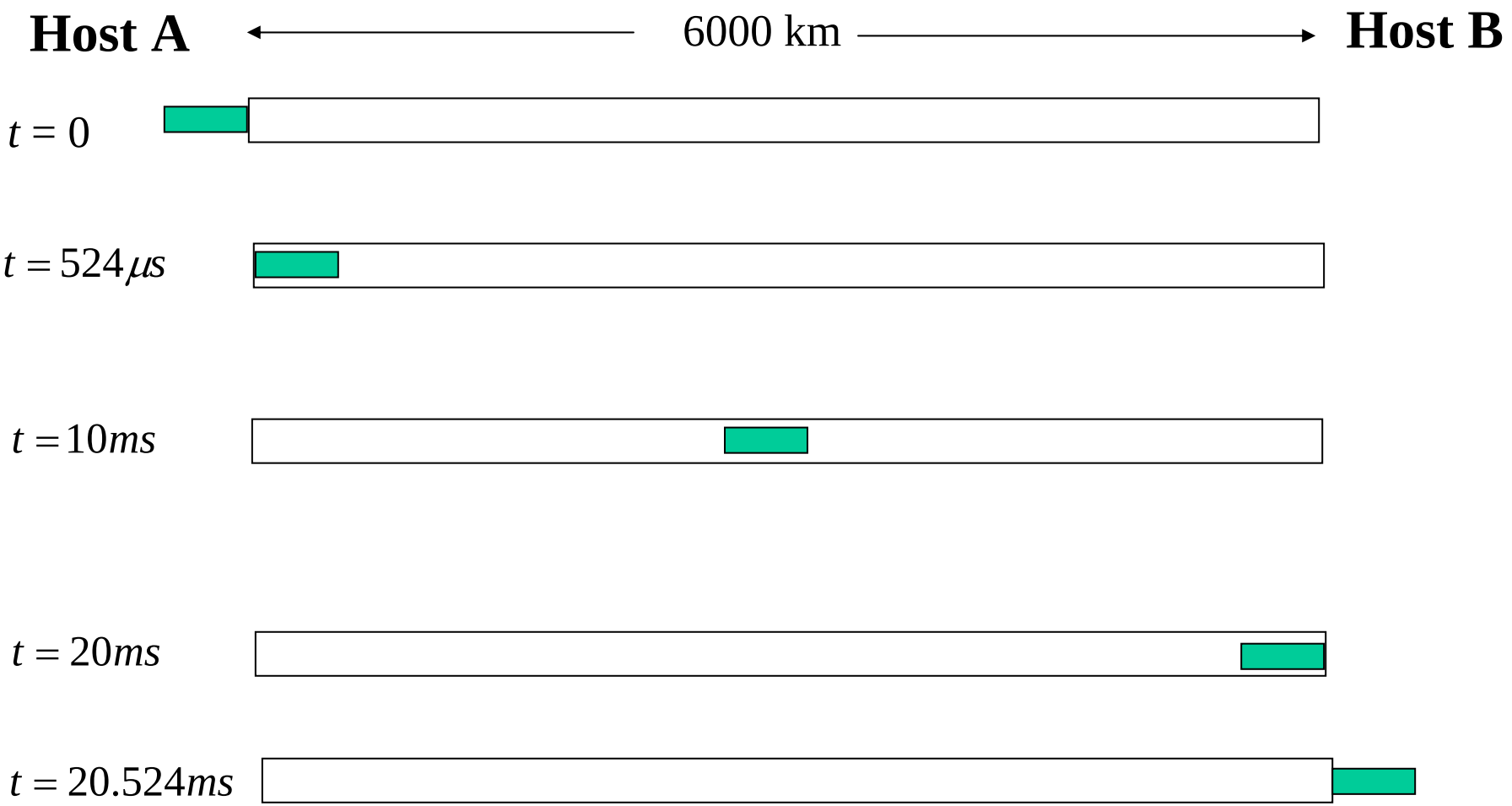
PERFORMANCE

- **E.g. An application on host A sends 1KB data to host B at time t0. Protocol adds 100 bytes overhead to the data. At time t0+1ms, the packet arrives at the transport layer of host B. Host B's buffer is full, so the packet is discarded. Host A times out an reTx the packet. At time t0+5ms, the packet is retrieved by the application at host B.**
  - (1) what is the throughput of the first transmission?
  - (2) what is the goodput?

PERFORMANCE

- **Bandwidth v.s. transmission delay**
  - Transmission delay (sec): determines by the propagation speed of electrical signal (v) and the distance between transmitter and receiver (d)
    - Has nothing to do with bandwidth
    - $t = d/v$
    - E.g. The distance between two places are 6000 km, the propagation speed is $3\times10^8 m/s$
      - The transmission delay is
  - Bandwidth (bps): determines how fast the data can be put in the channel by network interface card.
    - E.g. a packet is of size 64 KB. The channel bandwidth is 1Gbps. How long it will take the channel to put the packet in the channel?
      - 64 KB = 64 x 1024 x 8 = 524288 bits
      - $1\times10^9 bps$
      - $t = 524288/10^9 = 524\mu s$

PERFORMANCE



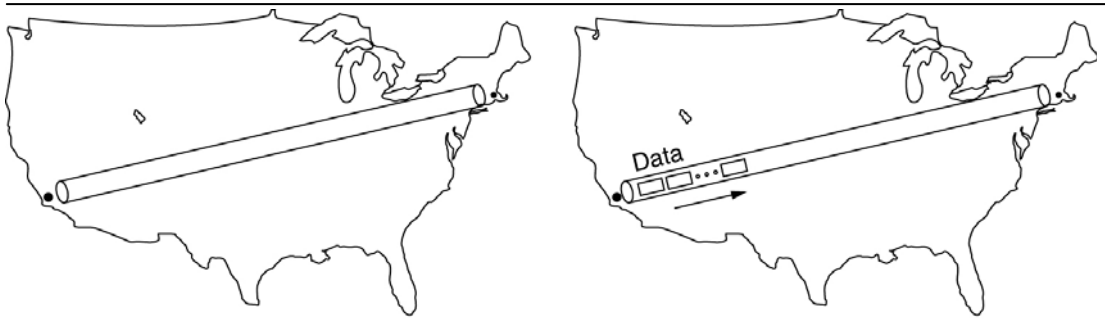
PERFORMANCE

- **Bandwidth-delay product (bits)**
  - Bandwidth (bps) x round trip delay (seconds)
  - Measures the capacity of the pipe from the sender to receiver and back.
    - How many bits are required to completely fill the pipe between transmitter and receiver at both directions.

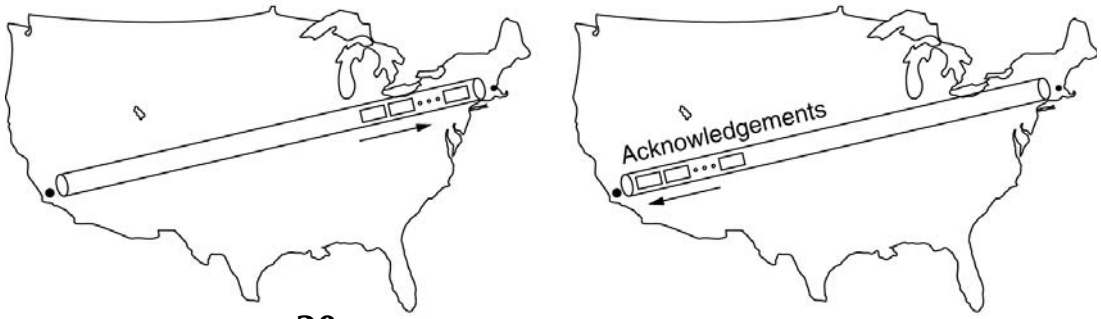


  - This value can be used to determine the window size for “fat” and “long” communication links
    - “Fat”: large bandwidth
    - “long”: long transmission delay.
    - volume of the pipe: Length x width (bandwidth-delay product)
    - How many bits are required to make full utilization of the high-speed network

PERFORMANCE



(a) **t=0** (b) **t=0.5ms**



(c) **t=20ms** (d) **t=40ms**

- transmission delay: 20 ms.
- Bandwidth: 1Gbps
- Window size: 64 KB.
- Bandwidth-delay product:
  - 1Gb x 40 ms = 40 million bits

- **Utilization:**
  - Transmitter use the transmission for 0.5 ms.
  - The second burst can be transmitted after 40 ms
  - 0.5/40 = 1.25% !!!

- **Solution: specify window size by bandwidth-delay product.**

SUMMARAY

- **Functions & Services**
  - First true end-to-end layer.
  - Connection-oriented reliable: TCP
  - Connectionless: UDP
- **Elements**
  - Major problem: delayed duplicate
  - Addressing, connection establishment (three-way handshake), connection release (symmetric release), flow control (variable window size), crash recovery
- **A simple transport protocol**
  - Implements of connection establishment, release, and data transmission.
- **UDP**
  - Addressing element. (no flow control, error control, etc.).
  - RTP (Real-time Transport Protocol): Provide basic functions required by most real-time applications. (E.g. timestamp, payload type)
- **TCP**
  - Header, connection establishment, connection release, flow control, congestion control, timer management.
- **Performance**
  - Throughput, delay, jitter, packet loss rate.
  - Bandwidth, throughput, goodput, bandwidth-delay product.